

WPA Laboratorium 8 -- wxWidgets

Wprowadzenie

- GUI: Graphical User Interface
- WIMP: Windows, Icons, Mice and Pop-up

GUI/WIMP umożliwia użytkownikowi interakcję z system operacyjnym z wykorzystaniem elementów graficzny wyświetlanych na ekranie. Interakcja ta jest możliwa za pomocą w myszy/piórka/dotyku, pozwalając „klikać” wybierać ikony i menu lub manipulować oknami.

Podstawą oprogramowania każdego środowiska w stylu WIMP jest **menedżer okien**. Menedżer okien kontroluje okna wyświetlane na ekranie. Obsługuje również przekazywanie informacji o zdarzeniach zachodzących w tych oknach do odpowiedniej aplikacji.

Okno to obszar ekranu, na którym może być wyświetlany tekst, grafika lub ich kombinację. Okna zazwyczaj mogą być **otwierane, zamykane, przesuwane i mieć zmieniany rozmiar**.

Ikona/Widget to mały obiekt graficzny, zwykle symbolizujący pewną operację taką jak otwarcie innego okna lub wywołanie funkcji.

Centralną częścią umożliwiającą interakcję użytkownika z takimi programami opartymi na WIMP jest **event loop**.

Jest to fragment menedżera okien który odbiera zdarzenia (events) wygenerowane przez widgety, takie jak kliknięcie przycisku przez użytkownika, wybór elementu menu lub wpisanie tekstu w polu. Gdy takie zdarzenie wystąpi, wykonywana jest powiązana z nim akcja (na przykład uruchomienie funkcji powiązanej z przyciskiem).

wxWidgets to darmowa, przenośna biblioteka GUI. Jest napisana w C++ i może zapewnić *natywny* wygląd i działanie na systemach operacyjnych takich jak Windows, Mac OS, Linux. wxPython to „wrapper” Pythona dla wxWidgets.

Pozwala on budować graficzne interfejsy użytkownika, używając typowych widgetów, takich jak **paski menu, menu, przyciski, pola, panele i ramki, pola tekstowe, radio button, check box**. W wxPython wszystkie elementy GUI są umieszczone w oknach najwyższego poziomu, takich jak `wx.Frame` lub `wx.Dialog`.

Te okna zawierają komponenty graficzne zwane **widgetami lub kontrolkami** (widgets/controls). Te widgety/kontrolki mogą być grupowane razem w panele (które mogą mieć lub nie posiadać reprezentację graficzną).

W wxPython możemy konstruować interfejsy użytkownika opierając się na:

- **Frame**: zapewnia podstawową strukturę okna: obramowanie, etykietę i kilka podstawowych funkcji (np. zmianę rozmiaru).
- **Dialog**: podobne do ramek, zwykle są to małe okna typu „Otwórz...”
- **Widget/Controls**: obiekty graficzne wyświetlane w ramce. Przykłady to **przyciski, pola wyboru, listy wyboru, etykiety i pola tekstowe**.
- **Containers**: grupują jeden lub więcej widgetów i/lub kontenerów

Więcej informacji na temat wxPython można znaleźć na stronach:

- Główna strona projektu: <https://www.wxpython.org/>
- Pełna Dokumentacja: <https://docs.wxpython.org/>
- Główne źródło wiedzy - dokumentacja modułu wx: <https://docs.wxpython.org/wx.1moduleindex.html>
- <https://docs.wxpython.org/gallery.html>
- <https://wiki.wxpython.org/Getting%20Started>

Zadanie

Które z wyróżnionych wyżej pojęć jesteś w stanie zidentyfikować w środowisku w którym pracujesz? Jakie inne elementy dostrzegasz?

Instrukcja

Zadania i przykłady proszę wykonywać równoległe z tekstem. Zadania proszę wykonywać jako skrypty, gdyż *Jupyter* i *wxWidgets* współpracują albo nie.

UWAGA: w dalszej części zakładamy, że biblioteka została zaimportowana w następujący sposób:

```
import wx
```

UWAGA: wx to nie jest standardowy moduł Pythona - trzeba go zainstalować w środowisku

Uwaga: Wybrane przykłady/zadania posiadają gotowe pliki umieszczone w katalogu `src`. W takim przypadku odpowiednia nazwa podana jest na początku kodu.

Uwaga: Poniższy tekst jest skrótem z dokumentacji, pomija wiele aspektów i dużo upraszcza. Na pierwsze spotkanie z *wxWidgets* może być, ale potem zalecam zapoznanie się podanymi udnosnikami.

Hello world!

Zadanie

Uruchomić poniższy program.

```
''' E01_hello_word.py
First wx app, simple frame with static text
'''

import wx

# Create the Application Object
app = wx.App()

# Now create a Frame (representing the top-level window)
frame = wx.Frame(parent=None, title=' Hello World')
# And add a text label to it
text = wx.StaticText(parent=frame, label= 'Hello Python')

# Display the window (frame)
frame.Show()

# Start the event loop
app.MainLoop()
```

Uwaga! Powinno pojawić się puste okienko, ale może być schowane pod innymi. Proszę poszukać go na pasku zadań!

Program tworzy nową instancję klasy `wx.App()` (Application Object) . Każdy dostarczający GUI z użyciem *wxPython* musi stworzyć jeden i tylko jeden 'Application Object'. W pewnym sensie jest to ekwiwalent funkcji `main()` w C++ - musi być, to tutaj starują okienka. Ten obiekt tworzy środowisko, odpowiada za zarządzanie kontrolkami oraz oknem aplikacji.

Obiekt klasy `wx.StaticText` odpowiada za wyświetlenie tekstu (jedna lub wiele linii) określonej przez parametr `label`. w naszym wypadku 'Hello Python'.

Obiekt `wx.StaticText` przyjmuje podczas tworzenia parametr `parent`, który określa jaki inny widget/kontener jest odpowiedzialny za jego umiejscowienie, narysowanie etc. W naszym przypadku jest to obiekt klasy `wx.Frame`.

Obiekt `wx.Frame` jako `parent` przyjmuje wartość `None` co oznacza, że jest on kontenerem najwyższego poziomu (top-level) i jest odpowiedzialny za pozostałe kontrolki/kontenery, zaś za niego odpowiada Application Object.

Zauważmy także, że po utworzeniu obiektu `frame` wywołana została funkcja `Frame.Show()` w celu wyświetlenia okna. Zwykle chcemy, aby główne okno programu było wyświetlone od razu po uruchomieniu. Jednak program może składać się z wielu okien, np. opcji dialogowych, które powinny być widoczne dopiero wtedy, gdy są potrzebne.

Ostatnim krokiem jest uruchomienie całej maszyny za pomocą funkcji `wx.App.MainLoop()`. Od teraz, ta funkcja przejmuje sterowanie aplikacją, przetwarzając eventy i inne formy interakcji z użytkownikiem aż do momentu zamknięcia okna.

The wx.App class

Klasa `wx.App` reprezentuje aplikację, jej przeznaczenie to:

- uruchomić całą maszynę związaną z *wxPython*
- „podłączyć się” do systemu operacyjnego i za pośrednictwem *natywnego windows managera* obsługiwać eventy i wyświetlać okno i pozostałe elementy GUI

Każdy program korzystający z wxPython musi mieć instancję wx.App. Powinna ona być otworzona jako pierwsza, przed innymi obiektami z biblioteki wx.

Zwykle tworzy się na jego bazie klasę potomną w celu dostosowania zachowania okna do potrzeb aplikacji. Głównie w celu podania własnych implementacji jednej z poniższych metod:

- OnPreInit: This method can be overridden to define behaviour that should be run once the application object is created, but before the OnInit method has been called.
- OnInit: This is expected to create the applications main window, display that window etc.
- OnRun: This is the method used to start the execution of the main program.
- OnExit: This can be overridden to provide any behaviour that should be called just before the application exits.

Nas będzie interesowała metoda OnInit, to tutaj będziemy tworzyć kontrolki i inne okna. Metoda ta zwraca wartość logiczną: True jeżeli nie wystąpiły błędy i można kontynuować tworzenie GUI albo False jeżeli zdażyło się nieszczęście (np. brakuje jakiegoś pliku), w tym wypadku program *bezpiecznie* się zakończy

Oto modyfikacja poprzedniego programu, w którym tworzymy własną klasę dziedziczoną z wx.App:

```
'''
E02_hello_word_bis.py
'''

import wx

class MyApp(wx.App):
    def OnInit(self):
        """ Initialise the main GUI Application"""

        # create a Frame (representing the top-level window)
        frame = wx.Frame(parent=None, title=' Hello World')

        # add a text label to it
        text = wx.StaticText(parent=frame, label= 'Hello Python')

        frame.Show()
        # Indicate whether processing should continue or not
        return True

if __name__ == "__main__":

    # This class can now be instantiated and the MainLoop started
    app = MyApp()

    # Run the GUI application
    app.MainLoop()
```

WWW: <https://docs.wxpython.org/wx.App.html>

Window Classes

The window or widget container classes that are commonly used within a wxPython application are:

- wx.Frame: A Frame is a top level window whose size and position can be set and can (usually) be controlled by the user.
- wx.Dialog: A Dialog is a top level window used for popups where the user has limited ability to interact with the window. In many cases the user can only input some data and/or accept or decline an option.
- wx.Panel: Is a container (non top level window) on which controls/widgets can be placed. This is often used in conjunction with a Dialog or a Frame to manage the positioning of widgets within the GUI.

wx.Frame

Dokumentacja: <https://docs.wxpython.org/wx.Frame.html>

Zadanie

Kliknąć w link i przeszkolować. To co widzimy to cała masa informacji technicznych dotyczących tej klasy, większość z nich jest obecnie bełkotem. Nas interesuje sekcja *Class API* a dokładnie konstruktor i niektóre argumenty.

```
__init__(self,
         parent,
         id=ID_ANY,
```

```

title="",
pos=DefaultPosition,
size=DefaultSize,
style=DEFAULT_FRAME_STYLE,
name=FrameNameStr)

```

- `parent` (`wx.Window`) - The window parent. This may be, and often is, `None`. If it is not `None`, the frame will be minimized, restored and *closed* when its parent is...
- `title` (string) - The caption to be displayed on the frame's title bar.
- `pos` (`wx.Point`) – The window position. The value `DefaultPosition` indicates a default position, chosen by either the windowing system or `wxWidgets`, depending on platform.
- `size` (`wx.Size`) – The window size. The value `DefaultSize` indicates a default size, chosen by either the windowing system or `wxWidgets`, depending on platform.

sub-zadanie: Czy rozumiesz *składnię* w powyższym fragmencie kodu?

Uwaga: zamiast klas `wx.Point` oraz `wx.Size` można użyć tuple:

```

* `pos = (x,y)`, w pikselach, liczone od lewego górnego rogu
* `size = (w,h)`, w pikselach

```

```

frame = wx.Frame(None,
    pos=(100,100),
    size = (400,400),
    title = "It's MY frame, and I will decide what the right position and size are.")

```

Zadanie

- Wykonaj kilka eksperymentów związanych z parametrami `pos` oraz `size`
- Dodaj w `OnInit` `frame.Center()`. Uruchom. Wyjaśnij.

UWAGA: Klasa `wx.Frame` jest klasą potomną kilku klas:

- `wx.TopLevelWindow`
- `wx.NonOwnedWindow`
- [wx.Windows](#)

Z tego powodu nie wszystkie dostępne metody są opisane bezpośrednio w dokumentacji `wx.Frame`. Dotyczy to każdego widgetu.

Zadanie

Klasa `frame` dziedziczy z `wx.Window` metodę `SetBackgroundColor(self, colour)`, jako `colour` można na podać trójkę (R,G,B) . Zmień kolor tła twojej aplikacji na taki, który lubisz.

Częstą praktyką jest tworzenie własnej klasy potomnej na bazie `wx.Frame`, z analogicznych powodów jak dla `wx.App`.

Example (see E03_MWE.py)

```

class MainFrame(wx.Frame):

    def __init__(self):

        # pass argument to the constructor as the base class
        super().__init__(parent=None, title='MWE', pos=(1000, 200), size=(400,400) )

        # create widgets
        # drink coffe if you like
        # but get things done

class MyApp(wx.App):

    def OnInit(self):
        """ Initialise the main GUI Application"""

        frame = MainFrame()
        frame.Show()

        return True

```

Klasa `MainFrame` jest klasą pochodną `wx.Frame` i jako tak dziedziczy wszystkie jej możliwości. W metodzie `__init__()` wywołujemy `super().__init__()` aby przekazać parametry takie jak rozmiar okna.

wx.Panel

Dokumentacja: <https://docs.wxpython.org/wx.Panel.html>

A panel is a window on which controls are placed. It is derived from wx.Window.

Zadanie

Dlaczego możemy być pewni, że konstruktor klasy wx.Widgets przyjmuje parametry pos and size oraz że dostępna jest funkcja SetBackgroundColour?

Klasa wx.Window udostępnia następującą funkcję:

```
SetSize (self, x, y, width, height, sizeFlags=SIZE_AUTO)
```

Sets the size of the window in pixels.

Parameters:

```
x (int) – Required x position in pixels, or DefaultCoord to indicate that the existing value should be used.  
y (int) – Required y position in pixels, or DefaultCoord to indicate that the existing value should be used.  
width (int) – Required width in pixels, or DefaultCoord to indicate that the existing value should be used.  
height (int) – Required height position in pixels, or DefaultCoord to indicate that the existing value should be used.
```

W następnym przykładzie używamy jej do umieszczania opieków klasy wx.Panel wewnątrz okna aplikacji (por. E04_frame_and_panels.py). Proszę zauważyć, że do konstruktora wx.Panel nie przekazujemy żadnych parametrów określających pozycję i rozmiar. Jedyny parametr przekazywany parametr self oznacza w tym kontekście, że „opiekunem” tej instancji wx.Panel jest klasa MainFrame.

```
class MainFrame(wx.Frame):  
  
    def __init__(self):  
  
        super().__init__(parent=None, title='04', pos=(1000, 200), size=(400, 400) )  
  
        # Set up the first Panel to be at position 1, 1 (the default)  
        # and of size 300 by 100  
        # with a blue background  
        self.top_panel = wx.Panel(self) # means parent = self i.e. an instance of MainFrame  
        self.top_panel.SetSize(1, 1, 150, 100)  
        self.top_panel.SetBackgroundColour(wx.Colour(0, 0, 255))  
  
        # Set up the first Panel to be at position 1, 100  
        # and of size 150 by 100  
        # with a blue background  
        self.center_panel = wx.Panel(self)  
        self.center_panel.SetSize(1, 100, 150, 100)  
        self.center_panel.SetBackgroundColour(wx.Colour(255, 0, 0))  
  
        # Set up the third Panel to be at position 150, 200  
        # and of size 150 by 100 with a green background  
        self.bottom_panel = wx.Panel(self)  
        self.bottom_panel.SetSize(150, 200, 150, 100)  
        self.bottom_panel.SetBackgroundColour(wx.Colour(0, 255, 0))
```

Zadanie

- Zakomentować linię self.top_panel.SetSize(1, 1, 150, 100). Uruchomić.
- Przekazać do konstruktora obiektu top_panel parametr size oraz pos z ulubionymi wartościami. Uruchomić.

Exercise

Try changing the size and position of top_panel. Then, experiment with the other panels: change their color and position. When you feel comfortable, add the fourth panel, and then arrange the panels in a chessboard pattern centered within the frame.

Widget/Control classes

W bibliotece wxWidgets dostępne są wszystkie podstawowe kontrolki jakich możemy oczekiwać w systemie okienkowym oraz sporo innych (patrz moduł wx.adv).

Przykładowo:

- wx.StaticText
- wx.StaticLine
- wx.StaticBox
- wx.Button/wx.ToggleButton/wx.RadioButton/wx.BitmapButton
- wx.TextCtrl
- wx.ListBox
- wx.CheckBox/wx.RadioButton
- wx.Choice
- wx.Combobox
- wx.Slider
- wx.SpinButton/wx.SpinCtrl
- wx.StaticBitmap

Zadanie

Przeskrolować stronę <https://docs.wxpython.org/gallery.html> w poszukiwaniu wyżej wymienionych widgetów.

Zadanie

Widgety są klasami potomnym wx.Control, która z kolei jest klasą potomną wx.Window. Jak zmienić ich rozmiar, pozycje i kolor?

Przyjrzymy się kilku wybranym widgetom

Uwaga: Whenever a widget is created it is necessary to provide the container window class that will hold it, such as a Frame or a Panel, for example:

```
enter_button = wx.Button(panel, label='Enter')
```

In this code snippet a wx.Button is being created that will have a label 'Enter' and will be displayed within the given Panel.

wx.StaticText

Dokumentacja: <https://docs.wxpython.org/wx.StaticText.html>

A static text control displays one or more lines of read-only text.

```
__init__(self, parent, id=ID_ANY, label="", pos=DefaultPosition, size=DefaultSize, style=0, name=StaticTextNameStr)
```

Constructor, creating and showing a text control.

Parameters:

```
parent (wx.Window) - Parent window. Should not be None.
label (string) - Text label.
pos (wx.Point) - Window position.
size (wx.Size) - Window size.
style (long) - Window style. See below
```

This class supports (among others) the following styles:

wx.ALIGN_LEFT: Align the text to the left.

wx.ALIGN_RIGHT: Align the text to the right.

wx.ALIGN_CENTRE_HORIZONTAL: Center the text (horizontally).

Zadanie

- Używając widgetu StaticText, dodaj na (mniej więcej)

- środku głównej ramki napis 'Napis na środku ramki'.
- prawym dolnym rogu górnej ramki napis Napis na dole ramki
- **Uwaga:** jeżeli w aplikacji jest tylko jeden widget, to nie pos i size może nie działać („bad boy!”. Ale jak są dwa widgety to już są grzeczne.

- Ustaw kolor tła na zielony.
- Zmień rozmiar widgetu na 80x80. Zmień na większy. Zmień na mniejszy. Co się dzieje z napisem?
- przekaż do konstruktora parametr style=wx.ALIGN_RIGHT a potem wx.ALIGN_CENTRE_HORIZONTAL

Zadanie

Napisz program którego okno wygląda tak jak obrazku src/rainbow_app.png

wxButton

Dokumentacja: <https://docs.wxpython.org/wx.Button.html#wx-button>

A button is a control that contains a text string, and is one of the most common elements of a GUI.

```
__init__(self, parent,
          id=ID_ANY,
          label="",
          pos=DefaultPosition,
          size=DefaultSize,
          style=0, validator=DefaultValidator, name=ButtonNameStr)
```

Parameters

parent (wx.Window) - Parent window. Must not be None.

id (wx.WindowID) - Button identifier. A value of ID_ANY indicates a default value.

label (string) - Text to be displayed on the button.

pos (wx.Point) - Button position.

size (wx.Size) - Button size. If the default size is specified then the button is sized appropriately for the text.

The preferred way to create standard buttons is to use default value of label.

If no label is supplied and id is one of standard IDs, a standard label will be used.

In other words, if you use a predefined ID_XXX constant, just omit the label completely rather than specifying it.

Some standard IDs (see https://docs.wxpython.org/stock_items.html#stock-items):

wx.ID_APPLY	wx.ID_CANCEL	wx.ID_CUT	wx.ID_NO
wx.ID_OK	wx.ID_SAVE	wx.ID_STOP	wx.ID_UNDO wx.ID_YES

Zadanie

- Umieść w GUI trzy przyciski za pomocą poniższych poleceń:

```
self.first_button = wx.Button(self, label='My first button', pos=(1,20))
self.OK_button = wx.Button(self, id=wx.ID_OK, pos=(1,60))
self.CANCEL_button = wx.Button(self, id=wx.ID_CANCEL, pos=(100,60))
```

- Poklikaj na przyciski. Co się dzieje?
- Naciśnij kilka razy klawisz Tab, naciśnij kilka raz klawisze strzałek. Co się dzieje?
- Dodaj jeszcze dwa przyciski: jeden z Twoją etykietą, a drugi z etykietą standardową

Interludium 1: grupowanie widgetów

Dotychczas umieszczaliśmy wszystkie widgety bezpośrednio na głównej ramce. Bardzo często widgety są grupowane w funkcjonalnie/logicznie/wizualnie powiązane grupy. Grupy te z kolei są często wykorzystywane wielokrotnie (np.; para StaticText-Button). Utworzymy taką grupkę na dwa sposoby korzystając z wx.Panel.

Popatrzmy na poniższy kod

```
self.widget_group_1 = wx.Panel(parent=self,size = (100,100),pos = (100,100))

# for visibility
self.group_1.SetBackgroundColour((100,100,0))

self.button_1 = wx.Button(parent = self,
                           label='Button 1',
                           pos=(1,1))

self.button_2 = wx.Button(parent = self.widget_group_1,
                           label='Button 2',
                           pos=(1,1))
```

Najpierw tworzymy Panel widget_group_1 w ramach głównej ramki o ustalonym rozmiarze i pozycji. Następnie dodajemy dwa przyciski:

- button_1 umieszczony jest na głównej ramce (parent=self) w lewym górnym rogu
- button_2 umieszczony jest na panelu widget_group_1 (parent=self.widget_group_1) również w lewym górnym rogu, **ALE** teraz pozycja widgetu jest określana w układzie współrzędnych Panelu a nie głównej ramki.

Zadanie

Co się stanie jeżeli ustawimy pos=(50,80) dla button_2?

Zadanie

Dodaj dwa kolejne przyciski z id=wx.ID_OK. Pierwszy na głównej ramce, a drugi na panelu widget_group_1. Umieść je poniżej i na prawo od już istniejących przycisków. Jeżeli trzeba powiększ rozmiar widget_group_1

Zadanie

Na prawo od widget_group_1 utwórz panel widget_group_2, który zawiera:

- StaticText umieszczony w górnej części
- dwóch egzemplarzy Button umieszczonych symetrycznie poniżej

Hint: dodanie koloru tła ułatwia orientację

Drugie podejście polega na utworzeniu własnej klasy dziedziczącej z wx.Panel. De facto, tworzymy własny widget.

Uwaga: Kolejne etapy są rozbite na dwa pliki

Na początek:

```
# E06_button_panel_1.py

class OurWidget(wx.Panel):

    def __init__(self,parent, pos=wx.DefaultPosition,size=wx.DefaultSize):
        super().__init__(parent,wx.ID_ANY,pos,size)

        self.SetBackgroundColour(WPA_COLORS['SKY BLUE'])

class SampleFrame(wx.Frame):

    def __init__(self):

        # pass argument to the constructor as the base class
        super().__init__(parent=None,title='Buttons', size=(400, 400),pos=(1000,100))

        # for reference
        self.ref_frame = wx.Panel(self,pos=(150,150),size=(100,100))
        self.ref_frame.SetBackgroundColour(WPA_COLORS['PINK'])

        # our widget with default size and position
        self.our_widget_1 = OurWidget(self)

        # our widget with given size and position
        self.our_widget_2 = OurWidget(self,size=(50,50),pos=(75,75))
```

Zadanie

- OurWidget dziedziczy z wx.Panel. Zmień kolor tła our_widget_2.
- Dodaj kilka kolejnych paneli OurWidget.
- Zmodyfikuj konstruktor OurWidget w taki sposób, aby możliwe było podanie koloru przy tworzeniu obiektu.

Wprowadzimy więcej życia w nasz widget:

```
# E06_button_panel_2.py

class OurWidget(wx.Panel):

    def __init__(self, parent,
                  color=WPA_COLORS['SKY BLUE'],
                  pos=wx.DefaultPosition):

        super().__init__(parent, wx.ID_ANY, pos, size = (220, 80))

        self.SetBackgroundColour(color)

        # pole tekstowe z komunikatem na górze
        self.message = wx.StaticText(self, label='MESSAGE', pos=(1, 1), size=(230, 30),
                                     style=wx.ALIGN_CENTRE_HORIZONTAL)

        # dwa przyciski na dole
        self.left_button = wx.Button(self, label='LEFT', pos=(5, 40), size=(100, 30))
        self.right_button = wx.Button(self, label='RIGHT', pos=(115, 40), size=(100, 30))
```

Zauważmy, że rozmiar widgetu jest obecnie ustalony, podobnie jak pozycja i rozmiar pola tekstowego i przycisków. Drugą ważną rzeczą jest, to że self przekazany do widgetów oznacza, teraz nasz widget. To on jest odpowiedzialny za pole tekstowe i przyciski.

Zadanie

- Dodaj kilka kolejnych paneli OurWidget.
- Zmodyfikuj konstruktor OurWidget w taki sposób, aby możliwe było podanie message przy tworzeniu obiektu.

Zadanie

Zadanie dodaj do OurWidget:

- kolejne pole tekstowe pomiędzy istniejącym polem tekstowym, a przyciskami
- kolejny przycisk center_button znajdujący się pomiędzy istniejącymi przyciskami

Zadanie

Stwórz własny widget, który przedstawia centralne pole tekstowe oraz cztery przyciski umieszczone na jego rogach.

Interludium 2: sizers

Zadanie

Uruchom ponownie program którego okno wygląda tak jak obrazku src/rainbow_app.png.

- Za pomocą myszy zmien pozycje okna programu. Co się stało?
- Zmaksymalizuj okno za pomocą ikonki na pasku tytułowym. Co się stało?
- Za pomocą myszy zmień rozmiar okna programu z poprzedniego zadania. Co się stało?
- Za pomocą myszki zmień rozmiar okna na jak najmniejszy. Co się stało?

Powyższe zadanie pokazuje dlaczego „hardcoded” położenie widgetów (i ogólnie używanie magic constants) nie jest dobrym pomysłem.

Rozwiązania są dwa:

- zabronić użytkownikowi zmiany rozmiaru okna, jest to możliwe przekazując odpowiednie parametry do konstruktora wx.Frame
- wyliczać pozycje widgetów relatywnie do rozmiaru okna i widgetów -- szerokości, wysokości i proporcji!

Zadanie

Zastanów się jaki byłby wzór na współrzędne widgetu, który miałby być utrzymywany dokładnie na środku okna.

Oba rozwiązania są stosowane przy budowaniu GUI. Przy czym rozwiązanie drugie jest zwykle realizowane za pomocą specjalnych konstrukcji/klas. Jedną z możliwości są tzw. *position anchors*. Drugą są siatki (są różne nazwy grid/layout). Są to specjalne klasy, które

- jest kontenerem odpowiedzialnym za pozycje widgetów, organizuje ich układ, np.: w jednym wierszu, na siatce
- po zmianie rozmiaru okna, na podstawie geometrii okna i warunków brzegowych (np. minimalny rozmiar) wyliczają pozycję widgetów

Przykład z `OurWidget` pokazuje ponadto, że ręczne wyliczanie pozycji widgetów jest, lekko mówiąc, uciążliwe. Można co prawda użyć funkcji typu `GetSize` (są takie), ale to rozwiązanie jest również uciążliwe (naprawdę). Ten problem również jest rozwiązywany przez (te same) pomocnicze klasy.

W bibliotece `wxWidget` ta koncepcja jest realizowana przez klasę `wx.Sizer`, a dokładnie przez jej klasy pochodne:

- opis zasady i przykłady https://docs.wxpython.org/sizers_overview.html#sizers-overview
- klasa bazowa `wx.Sizer` <https://docs.wxpython.org/wx.Sizer.html#wx-sizer>
- Zobacz także w Qt: <https://doc.qt.io/qt-6/qlayout.html>

Sizers, as represented by the `wx.Sizer` class and its descendants in the wxPython class hierarchy, have become the method of choice to define the layout of controls in dialogs in wxPython because of their ability to create visually appealing dialogs independent of the platform, taking into account the differences in size and style of the individual controls. Editors such as `wxDesigner`, `DialogBlocks`, `XRCed` and `wxWorkshop` create dialogs based exclusively on sizers, practically forcing the user to create platform independent layouts without compromises.

Uwaga: Tak, istnieją programy które pozwalają na tworzenie GUI metodą kliknij-przenieś i generujące odpowiedni kod. Tak, należy z nich korzystać tworząc dużą i/lub prawdziwą aplikację. Na etapie nauki dobrze jest jednak popracować bezpośrednio na kodzie, aby opanować nazewnictwo i łatwiej zrozumieć kod generowany przez narzędzia.

Nas będą interesować klasy:

- `wx.BoxSizer`: organizuje widgety liniowo: horyzontalnie lub wertykalnie <https://docs.wxpython.org/wx.BoxSizer.html#wx-boxsizer>
- `wx.GridSizer`: organizuje widgety w formie siatki <https://docs.wxpython.org/wx.GridSizer.html#wx-gridsizer>

wx.BoxSizer

Zacznijmy od następującego fragmentu

```
# E07_box_sizer.py
class SampleFrame(wx.Frame):

    def __init__(self):

        # pass argument to the constructor as the base class
        super().__init__(parent=None, title='Box sizer', size=(100, 100), pos=(1000, 100))

        self.grp_of_buttons = wx.BoxSizer(wx.HORIZONTAL)

        self.grp_of_buttons.Add( wx.Button(self, id=wx.ID_OK), 0, 0, 0 )

        self.our_button = wx.Button(self, id=wx.ID_CANCEL)
        self.grp_of_buttons.Add(self.our_button, 0, 0, 0 )
```

Pierwszym krokiem jest utworzenie obiektu `grp_of_buttons` klasy `wx.BoxSizer`, argument `wx.HORIZONTAL` oznacza, że widgety mają być ułożone poziomo. Dla ustawienia pionowego trzeba użyć `wx.VERTICAL`

Drugim krokiem jest dodanie widgetu do sizera za pomocą funkcji `Add`. Jej pierwszy argument to widget który ma być dodany. Pozostałe trzy argumenty ustawiamy na 0 i na razie o nich nie myślimy.

Mamy tu dwa sposoby:

- przekazanie widgetu bezpośrednio, jako argument funkcji
- utworzenie widgetu wcześniej osobno i przekazanie go potem do sizera

Zauważmy, że `parent` widgetów to nadal klasa `SampleFrame`.

Zadanie

- Uruchomić program `E07_box_sizer.py`. Co jest nie tak?
- Odkomentować linię oznaczoną jako (STEP1). Uruchomić ponownie. Jest różnica?
- Za pomocą myszy zmienić rozmiar okna na bardzo mały i bardzo duży. Co się dzieje?

Klasa `wx.Sizer` i jej potomne nie ma reprezentacji graficznej, nie ma też rozmiaru. Jej zadaniem jest przekazywanie do obiektu klasy `wx.Window` (lub potomnych) informacji o rozmiarze i położeniu widgetów.

Odkomentowanie linii

```
self.SetSizer(self.grp_of_buttons)
```

Ma mniej więcej następujący efekt:

- okno dowiaduje się, że ma pomocnika, który wie jak rozmieścić widgety
- po zmianie rozmiaru (w tym otwarciu) okno zapytuje `Sizer` o położenie widgetów, które są obserwowane przez `Sizer` -- te dodane za pomocą `Add`
- `Sizer` wylicza pozycje widgetów tak aby zmieściły się w oknie i przekazuje te informacje do okna
- jak widać, czasami efekt nie jest zadowalający

Zadanie

- Odkomentować linię oznaczoną jako (STEP2). Uruchomić ponownie. Jest różnica?
- Za pomocą myszy zmień rozmiar okna. Co się dzieje?

Odkomentowanie linii

```
self.grp_of_buttons.SetSizeHints(self)
```

Ma mniej więcej następujący efekt:

- `Sizer` dostaje wskazówki od okna na temat rozmiaru widgetów
- ustalony zostaje minimalny rozmiar okna w taki sposób aby wszystkie widgety się w nim zmieściły bez nachodzenia na siebie.

UWAGA W tym momencie podawanie parametru `size` jest zbędne.

Zadanie

- Usunąć argument `size` z konstruktora `SampleFrame`. Zauważ, że `Sizer` dostosował rozmiar okna.
- Odkomentować linie oznaczone jako (STEP3). Zauważ, że `Sizer` dostosował rozmiar okna.

Zadanie

- Zmień układ widgetów na poziomy.

Zadanie

- Zamiast jednego z przycisków dodaj pole tekstowe

Zadanie

- Zamiast jednego z przycisków dodaj widget który stworzyłeś na bazie `wx.Panel` w serii zadań dotyczących grupowania widgetów. **NIE RÓB COPY-PASTE. PRZECIEŻ UMIESZ KORZYSTAĆ Z MODUŁÓW!!!** (umiesz, nie? jeżeli nie to zobacz (STEP4), fajnie?)

wx.GridSizer

Obiekt `wx.GridSizer` rozmieszcza widgety na siatce.

```
# E07_grid_sizer.py
```

```
class SampleFrame(wx.Frame):

    def __init__(self):

        # pass argument to the constructor as the base class
        super().__init__(parent=None, title='Box sizer', size=(100, 100), pos=(100, 100))

        self.grp_of_buttons = wx.GridSizer(3, 3, 0, 0)

        self.grp_of_buttons.Add( wx.Button(self, id=wx.ID_OK), 0, 0, 0 )
```

Dwa pierwsze argumenty konstruktora `wx.GridSizer` to ilość wierszy i kolumn, dwa kolejne to odstępy pomiędzy kratkami.

Uwaga: Tutaj także stosują się `SetSizer` oraz `SetSizeHints`.

Zadanie

- Odkomentuj (STEP1). Zauważ, że rozmiar każdej kratki został dostosowany do rozmiaru `OurWidget`.

Zadanie

- Wykonaj kilka eksperymentów: zmień rozmiar siatki, dodaj/usuń różne widgety.

Bardziej złożony układ widgetów

Połączmy teraz nabyte wiadomości. Na początek umieścimy jeden sizer w drugim

```
class SampleFrame(wx.Frame):

    def __init__(self):

        # pass argument to the constructor as the base class
        super().__init__(parent=None, title='Box sizer', size=(100, 100), pos=(1000,100))

        self.outer_sizer = wx.BoxSizer(wx.VERTICAL)

        self.inner_sizer_1 = wx.BoxSizer(wx.HORIZONTAL)

        # add inner sizer as the first element of outer
        self.outer_sizer.Add(self.inner_sizer_1,0,0,0)

        # fill inner sizer with widgets
        self.inner_sizer_1.Add( wx.Button(self, id=wx.ID_OK),0,0,0 )
        self.inner_sizer_1.Add( wx.Button(self, id=wx.ID_APPLY),0,0,0 )
        self.inner_sizer_1.Add( wx.Button(self, id=wx.ID_CANCEL),0,0,0 )
```

Zadanie

Uruchom `E07_more_sizers.py`. Czy efekt jest taki jak oczekiwany?

Ciekawszy efekt uzyskamy gdy utworzymy kolejny wewnętrzny sizer `inner_sizer_2` i dodamy go do `outer.sizer`.

Zadanie

Odkomentuj (STEP1) i uruchom.

Zadanie

- Dodaj do `inner_sizer_1` kolejny przycisk.
- Dodaj kolejny sizer wewnętrzny `inner_sizer_3` który zawiera dwa widgety `OurWidget` (odkomentuj (STEP2))

Teraz stworzymy własny widget korzystający z sizera:

```
# E08_complex_widget.py
class RowOfButtons(wx.Panel):

    def __init__(self, parent, labels):
        super().__init__(parent, wx.ID_ANY, pos, size)

        self.sizer = wx.BoxSizer(wx.HORIZONTAL)

        for l in labels:
            self.sizer.Add( wx.Button(self, label=l),0,wx.ALL,5)

        self.SetSizer(self.sizer)
```

Jest to kasa pochodna `wx.Panel` (już to robiliśmy!). Jej zadaniem jest wygenerowanie na podstawie podanego zestawu etykiet wiersz zawierający przyciski o tych etykietach. Jest to realizowanie z pomocą `wx.BoxSizer`, który organizuje przyciski we wierszu. Same przyciski są tworzone i przypisywane do sizer w petli `for`. Zauważmy, że w ostatniej linijce widget został poinformowany o tym, że posiada sizer.

Użycie tego widgetu w aplikacji jest bardzo proste:

```
# E08_complex_widget.py
class SampleFrame(wx.Frame):

    def __init__(self):
```

```
# pass argument to the constructor as the base class
super().__init__(parent=None, title='Box sizer', size=(100, 100), pos=(1000, 100))

self.outer_sizer = wx.BoxSizer(wx.VERTICAL)

self.outer_sizer.Add(RowOfButtons(self, '123456'), 0, 0, 0)
self.outer_sizer.Add(RowOfButtons(self, 'ABCDEFGH'), 0, 0, 0)
self.outer_sizer.Add(RowOfButtons(self, ['Ala', '0la', 'Ela', 'Ula', 'Ila', 'Yla']), 0, 0, 0)

self.outer_sizer.SetSizeHints(self)
self.SetSizer(self.outer_sizer)
```

Zadanie

Stwórz własny widget wyglądający jak klawiatura numeryczna.
